



Moldable and castable polygons[☆]

David Rappaport, Arnold Rosenbloom*

Computing and Information Science, Queens University, Kingston Ontario, Canada K7L 3N6

Communicated by Subhash Suri; submitted 6 August 1992; accepted 6 January 1994

Abstract

This paper introduces the concepts of moldability and castability of simple polygons and relates moldability to monotonicity. We detail a $\Theta(n)$ algorithm for determining all n forward maximal monotone chains of a simple polygon and apply this algorithm to the problems of determining 2-moldability, 2-castability and the minimum monotone decomposition of a simple polygon (Swaminathan et al., 1992). Our results include a simple optimal algorithm solving the minimum monotone decomposition problem, an optimal algorithm to determine 2-moldability and an $O(n \log n)$ algorithm to determine 2-castability.

Key words: Monotonicity; Decomposition; Moldability; Castability

1. Introduction

A statue can be created by pouring wet cement into a mold, allowing the cement to harden, then removing the mold. If as well, the two pieces of the mold can be removed by translation then the resulting statue is 2-moldable. Alternatively, we can form the statue by taking two cast pieces, lie them on their sides, fill them up with cement and after the cement hardens glue the pieces together along the flat sides. Of course we still want to remove the cast pieces without breaking them. A statue that can be created in such a way is a 2-castable statue.

This paper introduces the notions of moldability and castability of simple polygons and investigates the 2-moldability/2-castability of simple polygons. Our results include theory relating 2-moldability to 2-monotonicity and a $\Theta(n)$ time and space

[☆]This research was supported in part by the Natural Sciences and Engineering Research Council of Canada under grant A-9204.

*Corresponding author.

algorithm for determining all forward maximal monotone chains of a simple polygon. Two immediate applications of this algorithm are determining the 2-moldability of a polygon and a minimum monotone decomposition [9] for a polygon. In both cases we exhibit a $\Theta(n)$ algorithm. Finally we use the results of [3] to obtain an $O(n \log n)$ algorithm for determining 2-castability.

The concept of monotonicity of chains of a polygon will play an important role in the development of algorithms for recognizing moldable sets. In [6] an $\Theta(n)$ time and space algorithm is given for determining if a polygon is monotone. In [5] a simple $\Theta(n)$ algorithm is given which triangulates a monotone polygon. In [9] an $\Theta(n)$ time and space algorithm is given (using the results of [4]) which determines the minimum number of monotone chains into which a given polygon can be decomposed. We provide a straightforward algorithm which solves the same problem in the same time and space bounds.

2. Notation

By a direction in $\mathbb{R}^2$ we mean a non-origin point which will be used to determine changes in position. By $\vec{d}$ we mean the ray originating at z and parallel to the direction d . Given 2 points p, q by the ray $\overrightarrow{pq}$ we mean the set of points on the half line based on p and including q . For any $X \subseteq \mathbb{R}^2$ ∂X , X° , X^c , $\bar{X}$ will denote the boundary of X , the interior of X , the complement of X and the closure of X . By an ε neighborhood of z we mean the set of points less than ε distance from z . $\mathbb{Z}_n$ denotes the integers modulo n and $\mathbb{Z}_n^{i,j}$ denotes the set of equivalence classes from i to j modulo n (i.e. $\{i \bmod n, (i+1) \bmod n, \dots, j \bmod n\}$). A polygon (see Fig. 1) is a finite sequence of segments (edges) which intersect only at their end points (the vertices) and only consecutive edges intersect.

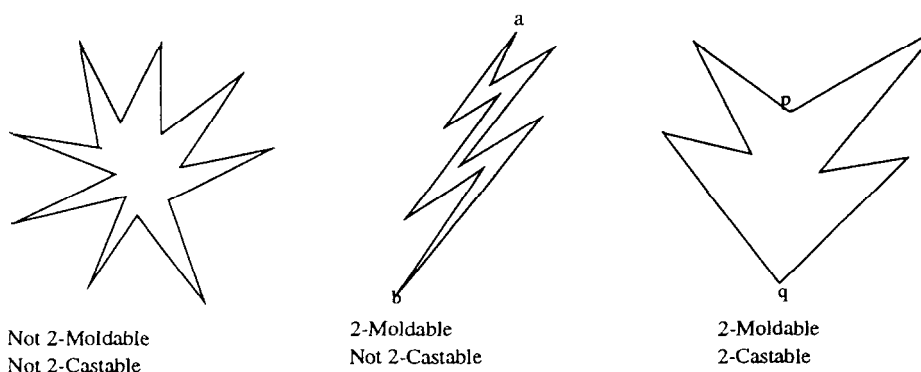


Fig. 1. Some example polygons.

Polygon Convention. The vertices are ordered so that the interior of the polygon lies to the left of the edges.

That is, if z is internal to some edge of the polygon P then there is a neighborhood of z so that all points in the neighborhood and to the left of the edge are contained in P° .

Given 2 points $p, q \in \partial P$ by $(p \cdots q)$ we mean the set of boundary points of P between p and q in the natural direction. $[p \cdots q] = (p \cdots q) \cup \{p, q\}$. If v and v' (appearing in this order) are sequential vertices of the chain $[p \cdots q]$ then $v' - v$ is a vector induced by $[p \cdots q]$. Note that we consider p and q vertices of $[p \cdots q]$.

2-Moldability will be in some sense the most general type of moldability. It corresponds to being able to fill up a set while a mold is in place and then being able to remove the mold by translation without breaking it.

We will say that the chain $[p \cdots q]$ of polygon P is *removable* in direction d precisely when $\forall z \in [p \cdots q] \quad \overset{d}{\rightarrow} z \cap P^\circ = \emptyset$.

Definition. A polygon P is a *2-moldable* when there are chains $[p \cdots q], [q \cdots p]$ and directions d_{pq}, d_{qp} such that $[p \cdots q]$ is removable in direction d_{pq} and $[q \cdots p]$ is removable in direction d_{qp} .

An obvious extension of the above definition is to the definition of k -moldability. Polygon P is *k-moldable* if there is some partition of the boundary into k pieces such that each piece is removable in some direction.

3. Polygons and monotonicity

A chain $[p \cdots q]$ of a polygon P is *monotone with respect to L* if there is some orientation of L so that the ordering of the projection of the vertices in $[p \cdots q]$ on L agrees with the ordering of the vertices themselves. The chain $[p \cdots q]$ is *monotone* if there is some line L so that $[p \cdots q]$ is monotone with respect to L . A *monotone decomposition* of a polygon is a decomposition of its boundary into disjoint (except at the endpoints) monotone chains. A polygon is *k-monotone* if there is a monotone decomposition of the polygon consisting of k chains.

It should be obvious that a k -moldable polygon is also a k -monotone polygon, but the converse is not in general true. The polygon in Fig. 2 is 3-monotone but is not

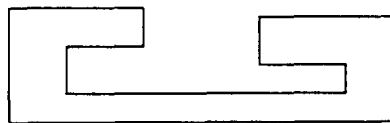


Fig. 2. A 3-Monotone polygon which is not k-moldable.

k -moldable for any k . To see this, note that the segment $\overline{pq}$ can not be translated indefinitely in any direction without intersecting the interior.

4. Towards a computable characterization of 2-moldability

Although general k -moldability and k -monotonicity are not equivalent, we can establish a relationship for the special case $k = 2$.

Lemma 1. *If $[p \cdots q]$ is removable in direction d then $[p \cdots q]$ is monotone with respect to a line L with L perpendicular to d .*

Corollary 2. *If polygon P is k -moldable then it is k -monotone.*

Lemma 3. *If $[p \cdots q]$ is monotone with respect to a line L_p and $[q \cdots p]$ is monotone with respect to a line L_q then there are directions d_{pq} and d_{qp} , perpendicular to L_p and L_q respectively, so that $[p \cdots q]$ is removable in direction d_{pq} and $[q \cdots p]$ is removable in direction d_{qp} .*

Proof. Without loss of generality let p and q lie on a vertical line L with q above p . We are free to orient the directions d_{pq} and d_{qp} so that d_{pq} points to the right and d_{qp} points to the left of L . Since the chain $[p \cdots q]$ is monotone with respect to a line perpendicular to d_{pq} , it follows that $[p \cdots q]$ is constrained to lie within a strip bounded by lines through $\overset{d_{pq} \rightarrow}{p}$ and $\overset{d_{pq} \rightarrow}{q}$. Similarly $[q \cdots p]$ is constrained to lie within a strip bounded by the lines through $\overset{d_{pq} \rightarrow}{p}$ and $\overset{d_{pq} \rightarrow}{q}$. As a result no part of P can cross L below the point p or above the point q .

We show that $\overset{d_{pq} \rightarrow}{z}$ does not intersect P° for all z in $[p \cdots q]$. For suppose that $\overset{d_{pq} \rightarrow}{z}$ does intersect P° . If $\overset{d_{pq} \rightarrow}{z}$ intersects P° then $\overset{d_{pq} \rightarrow}{z}$ must also intersect $(q \cdots p)$ because $[p \cdots q]$ is monotone with respect to a line perpendicular to direction d_{pq} . Let x denote a point on $\overset{d_{pq} \rightarrow}{z}$ intersecting $(q \cdots p)$. There is a continuous path from x to p in $[p \cdots q]$ and a continuous path from x to q in $[q \cdots p]$. However, one of these paths must either cross L below p or above q , a contradiction. Therefore, $\overset{d_{pq} \rightarrow}{z}$ cannot intersect $(q \cdots p)$ and we conclude that $[p \cdots q]$ is removable in direction d_{pq} . A similarly symmetric argument can be used to show that $[q \cdots p]$ is removable in direction d_{qp} . $\square$

Note that if a polygon is 2-monotone (and so 2-moldable) then there are a pair of vertices which determine a 2-monotone (and hence a 2-moldable) decomposition for the polygon. So in order to determine the 2-moldability of a polygon we can examine vertex pairs v_i, v_j to see if both $[v_i \cdots v_j]$ and $[v_j \cdots v_i]$ are monotone. Unfortunately, this naive algorithm has complexity at least $O(n^2)$. We develop a linear time algorithm in the next section.

5. Computing forward maximal monotone chains

This section details a $\Theta(n)$ time and space algorithm for determining all forward maximal monotone chains for a given polygon. As well as being of interest in its own right, this algorithm will be used to develop $\Theta(n)$ algorithms for determining 2-monotonicity, 2-moldability and a minimum monotone decomposition of a simple polygon. A further application of our work here will be in establishing a $O(n \log n)$ algorithm for determining 2-castability.

5.1. Computing $C[]$

A chain $[p \cdots q]$ is *forward maximally monotone* if it is monotone but any extension of the chain (at the q end) produces a non-monotone chain. Our efforts throughout this section will be in detailing an algorithm which, given a polygon as a sequence of its n vertices, determines the array $C[]$ which describes all the forward maximal monotone chains for the polygon. That is, $C[]$ satisfies $C[i] = j$ if and only if $[v_i \cdots v_j]$ is a forward maximal monotone chain. We arrive at the algorithm *Compute $C[]$* by noticing that there is an intimate connection between the monotonicity of a chain $[p \cdots q]$ and the directions determined by edges in the chains. The connection allows us to rephrase our problem of determining all forward maximal monotone chains to a problem about points on a circle. An optimal solution to this final problem is obtained by noticing some properties of the problem structure which allow a simple linear time algorithm.

Lemma 4. *A chain $[p \cdots q]$ of P is monotone if and only if all vectors induced by the chain lie in some closed half-plane through the origin.*

By the *unit circle* $\bigcirc$ we will mean $\{(\cos(\theta), \sin(\theta)) \mid \theta \in \mathbb{R}\}$. For $a, b \in \bigcirc$, by the *angle from a to b* , written $\angle ab$, we will mean the set of points of $\bigcirc$ from a counterclockwise to b . The *acute angle between a and b* , written $\angle \angle ab$, will be the angle created by the smaller of $\angle ab$ and $\angle ba$. We will denote the size of $\angle ab$ by $|\angle ab|$. The reader should realize that there is a natural mapping between the real interval $[0, 2\pi)$ and points on the unit circle. Either representation may be used but our algorithms will assume the $[0, 2\pi)$ representation. The array $C[]$ is computed by mapping edges $\overline{v_i v_{i+1}}$ to points $c_i = 0\vec{d_i} \cap \bigcirc$ (where $d_i = v_{i+1} - v_i$) and solving the equivalent problem.

Instance: A set of points $\{c_0, \dots, c_{n-1}\}$ on the unit circle which satisfies:

1. $0 < |\angle c_i c_{i+1}| < \pi$.
2. The complete set $\{c_0, \dots, c_{n-1}\}$ does not fit in any angle of size π .

Problem: For each $i \in 0, \dots, n-1$ determine $P[i]$ which satisfies:

1. The set of points $\{c_i, \dots, c_{P[i]}\}$ fits in some angle of size π .
2. The set of points $\{c_i, \dots, c_{P[i]}, c_{P[i]+1}\}$ does not fit in any angle of size π .

The above restrictions on the instance arise as a result of the mapping from sequential vertex differences to points on the unit circle.

$\mathbb{Z}_n$ denotes the integers modulo n and $\mathbb{Z}_n^{i,j}$ denotes the set of equivalence classes from i to j modulo n (ic. $\{i \bmod n, (i+1) \bmod n, \dots, j \bmod n\}$). In what follows $i, j, a, b \in \mathbb{Z}_n$. For $a \neq b$

$$\angle_a^b = \cup_{k \in \mathbb{Z}_n^{a,b-1}} \angle c_k c_{k+1} \quad \text{and} \quad U_a^b = \{c_k | k \in \mathbb{Z}_n^{a,b}\}.$$

It should be clear that U_a^b fits in an angle of size π if and only if $\angle_a^b$ does and $\angle_a^b$ is the smallest angle containing U_a^b . An easy induction establishes that $\angle_a^b \neq \emptyset \Rightarrow \angle_a^b = \angle cc'$ for some $c, c' \in U_a^b$.

It is important to note that we are distinguishing points c_i, c_j not only as points of $\bigcirc$ but also by their indices. So for example, if $i \neq j$ then c_i and c_j might represent the same point of $\bigcirc$ but we will say they are different. To be precise, one should think of each c_i as an ordered pair (θ_i, i) where θ_i is a point on $\bigcirc$ and $i \in \mathbb{Z}_n$. We will say that c_j dominates c_i with respect to $\mathbb{Z}_n^{a,b}$ if j appears after i in the sequence $\langle a, a+1, \dots, b \rangle$.

Finally, we define the *first* and *last* members of a subset S of U_a^b as follows. $\text{first}(S)$ is the most dominant member of the set $\{c \in S | \forall d \in S \angle cd \subseteq \angle_a^b\}$. Similarly, $\text{last}(S)$ is the most dominant member of $\{c' \in S | \forall d \in S \angle dc' \subseteq \angle_a^b\}$. Thus $\angle \text{first}(U_a^b) \text{last}(U_a^b) = \angle_a^b$.

Our algorithm uses the quantities *first* and *last* to keep track of $\angle_a^b$. For the sake of algorithmic efficiency, we use a subset $\mathcal{L}_a^b \subseteq U_a^b$, so that $\text{first}(U_a^b) = \text{first}(\mathcal{L}_a^b)$ and $\text{last}(U_a^b) = \text{last}(\mathcal{L}_a^b)$ as points on the circle.

More formally

Algorithm: Compute P

```

 $\mathcal{L} \leftarrow \{c_0, c_1\}$ 
 $i \leftarrow 0 \quad j \leftarrow 1$ 
while ( $i < n$ ) do
  while ( $|\angle \text{first}(\mathcal{L}) \text{last}(\mathcal{L})| \leq \pi$ ) do
     $j \leftarrow j + 1 \bmod n$ 
     $S \leftarrow \{c_k \in \mathcal{L} : c_k \in \angle c_{j-1} c_j\}$ 
     $\mathcal{L} \leftarrow \mathcal{L} - S \cup \{c_j\}$ 
  end while
  if ( $\text{first}(\mathcal{L}) = c_j$ ) then  $k \leftarrow \text{index}(\text{last}(\mathcal{L}))$  else  $k \leftarrow \text{index}(\text{first}(\mathcal{L}))$ 
  while ( $i \leq k$ ) do
     $P[i] \leftarrow j - 1$ 
     $\mathcal{L} \leftarrow \mathcal{L} - \{c_i\}$ 
     $i \leftarrow i + 1$ 
  end while
end while
end Compute  $P$ 
```

5.1.1. Analysis and correctness

We begin by proving the correctness of Algorithm Compute P . We will use the notation $\mathcal{L}_a^b$ to denote the state of the set $\mathcal{L}$ when $i = a$ and $j = b$.

We must show that $\angle_a^b = \angle \text{first}(\mathcal{L}_a^b) \text{last}(\mathcal{L}_a^b)$.

Lemma 5. *If $\angle \text{first}(\mathcal{L}_a^b) \text{last}(\mathcal{L}_a^b) = \angle \text{first}(U_a^b) \text{last}(U_a^b) = \angle_a^b$ then*

$$\angle \text{first}(\mathcal{L}_a^{b+1}) \text{last}(\mathcal{L}_a^{b+1}) = \angle \text{first}(U_a^{b+1}) \text{last}(U_a^{b+1}) = \angle_a^{b+1}.$$

Proof. Recall that $\mathcal{L}_a^{b+1} = \mathcal{L}_a^b - \{c_k \in \mathcal{L}_a^b : c_k \in \angle c_{b-1} c_b\} + \{c_{b+1}\}$. We consider two cases. Suppose $c_{b+1} \in \angle_a^b$ then

$$\text{first}(\mathcal{L}_a^{b+1}) = \text{first}(\mathcal{L}_a^b) = \text{first}(U_a^b) = \text{first}(U_a^{b+1})$$

and

$$\text{last}(\mathcal{L}_a^{b+1}) = \text{last}(\mathcal{L}_a^b) = \text{last}(U_a^b) = \text{last}(U_a^{b+1})$$

and we are done. On the other hand suppose that $c_{b+1} \notin \angle_a^b$. In this case c_{b+1} either becomes the new value for $\text{first}(\mathcal{L}_a^{b+1})$ or $\text{last}(\mathcal{L}_a^{b+1})$. Either way we maintain $\text{first}(\mathcal{L}_a^{b+1}) = \text{first}(U_a^{b+1})$ and $\text{last}(\mathcal{L}_a^{b+1}) = \text{last}(U_a^{b+1})$. $\square$

Lemma 6. *If $\angle \text{first}(\mathcal{L}_a^b) \text{last}(\mathcal{L}_a^b) = \angle \text{first}(U_a^b) \text{last}(U_a^b) = \angle_a^b$ then*

$$\angle \text{first}(\mathcal{L}_{a+1}^b) \text{last}(\mathcal{L}_{a+1}^b) = \angle \text{first}(U_{a+1}^b) \text{last}(U_{a+1}^b) = \angle_{a+1}^b.$$

Proof. Recall that $\mathcal{L}_{a+1}^b = \mathcal{L}_a^b - \{c_a\}$. There are two cases to consider. If $c_a \in \angle_a^b$ then we are done. Otherwise when we remove c_a from $\mathcal{L}_a^b$ we will obtain a new value for $\text{first}(\mathcal{L}_{a+1}^b)$ or $\text{last}(\mathcal{L}_{a+1}^b)$. We first consider the case when $c_a = \text{first}(\mathcal{L}_a^b) = \text{first}(U_a^b)$. Use c_x to denote the value of $\text{first}(U_{a+1}^b)$. We must demonstrate that $c_x \in \mathcal{L}_a^b$. Any element c_k an element of U_a^b and not an element of $\mathcal{L}_a^b$ has the property that $c_k \in \angle c_{j-1} c_j$, and c_k is encountered prior to c_{j-1} in the while loop of algorithm Compute P . Since c_x is adjacent to c_a the only pair that contains c_x in U_a^b is $\angle c_a c_{a+1}$, but $x \notin \mathbb{Z}_n^{1,a}$ so c_x was not encountered prior to c_a in the while loop of algorithm Compute P . Therefore, c_x must also be a member of $\mathcal{L}_a^b$. We conclude that $c_x \in \mathcal{L}_{a+1}^b$ and that $\text{first}(\mathcal{L}_{a+1}^b) = \text{first}(U_{a+1}^b)$. A similar symmetric argument can be made to show that $\text{last}(\mathcal{L}_{a+1}^b) = \text{last}(U_{a+1}^b)$. $\square$

Consider the initial conditions of $a = 0$ and $b = 1$, and observe that $\angle_a^b = \angle \text{first}(\mathcal{L}_a^b) \text{last}(\mathcal{L}_a^b)$. Thus we can bootstrap an inductive argument based on the previous two lemmas to prove the correctness of algorithm Compute P . We now discuss the complexity of this algorithm.

Lemma 7. *Algorithm Compute P can be implemented in $O(n)$ time and space.*

Proof. The key to an efficient implementation of Compute P is to use a suitable data structure to represent the set $\mathcal{L}$. We maintain the values of $\mathcal{L}$ in a doubly linked list

sorted according to their $\angle$ values. That is, following an element's forward link leads to that elements nearest counterclockwise neighbour. The list is also maintained so that $\text{first}(\mathcal{L})$ and $\text{last}(\mathcal{L})$ appear as the first and last element of the list. The principle difficulty in obtaining the linear running time is in updating the representation of $\mathcal{L}_i^j$ to $\mathcal{L}_i^{j+1}$, and updating $\mathcal{L}_i^j$ to $\mathcal{L}_{i+1}^j$. We address the problem of updating $\mathcal{L}_i^j$ to $\mathcal{L}_i^{j+1}$ first. To remove the set $\{c_k \in \mathcal{L} : c_k \in \angle c_j c_{j+1}\}$ we traverse the list from the location of c_j to the location of c_{j+1} deleting every element we encounter. Furthermore we keep track of the location of the most recent addition to $\mathcal{L}$. Since any time we transverse an element we also delete it, the accumulated effort throughout the life of the algorithm is $O(n)$.

In updating $\mathcal{L}_i^j$ to $\mathcal{L}_{i+1}^j$ we must determine the location of c_i in $\mathcal{L}$. We claim that c_i is either $\text{first}(\mathcal{L})$, $\text{last}(\mathcal{L})$, or not in $\mathcal{L}$. To see why this is so, consider c_i not first or last such that $c_x = \text{first}(\mathcal{L})$ and $c_y = \text{last}(\mathcal{L})$ and $i < x < y$. (Note we are only concerned with the fact that $i < x$ and that $i < y$, the relative ordering of x and y is not important, we have picked arbitrarily.) Since $x < y$ then at some time in the process of updating the set $\mathcal{L}$ we scanned the circular list from c_x to c_y . Thus for all elements c_i such that $i < x < y$, we know that c_i cannot be in $\mathcal{L}$. Therefore, the cost that has been incurred is bounded from above by a constant.

Thus both updates are taken care of. We are obviously within the $O(n)$ bound for space complexity. Thus we conclude that algorithm *Compute P* has a time and space complexity in $O(n)$. $\square$

Returning to our original problem of computing maximal monotone chains observe that it follows from Lemma 4 that $C[i] = P[i] + 1$. Coupled with our algorithm analysis the previous statement leads to the main result of this section.

Theorem 8. *Algorithm Compute P can be used to compute maximal monotone chains in $O(n)$ time and space.*

We will refer to the algorithm in the above theorem as *Compute C* .

5.2. Application 1: determining 2-moldability of a polygon

We can now use *Compute C* to determine if a polygon is 2-moldable/2-monotone.

Algorithm: *Is 2-moldable?*

for $i \in \langle 0, \dots, n-1 \rangle$

 if $C[C[i]]$ appears in $i, i+1, \dots, C[i]$ (all numbers taken modulo n) then result(moldable)

end for

result(notmoldable)

Theorem 9. *There is a linear time and space algorithm which, given a polygon P as an array $v[]$ of n vertices, determines if P is 2-moldable and if so returns a 2-mold for P .*

5.3. Application 2: minimum monotone decomposition

We can also apply our $C[]$ algorithm to the problem of [9] and solve the minimum monotone decomposition problem. That is, given a polygon as a sequence of vertices, determine its minimum (in number of chains) monotone decomposition. The algorithm presented below as well as our Compute $P[]$ algorithm first appeared in [7, 1] and was later adopted as an improvement to [8] in the paper [9].

Consider the problem of finding a minimum cover for a circle given the set of n arcs $\angle z_i z_{C[i]}$ where $z_i \in \bigcirc$ has coordinates $(\cos(i2\pi/n), \sin(i2\pi/n))$. We can use a queue to provide the algorithm of [4] with an ordered sequence of arc endpoints and find a minimum cover $\angle z_{i_1} z_{C[i_1]}, \dots, \angle z_{i_m} z_{C[i_m]}$ for $\bigcirc$. We claim that this minimum circle cover corresponds to a minimum monotone decomposition of P and that the algorithm takes linear time and space, the same bounds as [9]. It should be noted that some variant of the algorithm of [9] might be used to decide 2-moldability.

We compare our results with the results of [9] by comparing both as algorithms which solve the Minimum Monotone Decomposition problem. The approach in [9] is to suggest that once all maximal monotone chains are obtained in sorted order then the algorithm of [4] solves the decomposition problem in linear time. They then break the universe of all polygons into 2 disjoint sets. The first admits a simple greedy solution to the decomposition problem. For the second set, they supply an algorithm (which applies only to this set) for determining all maximal monotone chains in sorted order. Once this is established, the comments regarding [9] implies the linear time result for polygons in the second set. These two results, together with an algorithm for determining membership in the sets, from their decomposition algorithm.

Our approach does not distinguish polygons. For any polygon, we compute all maximal monotone chains and then use the algorithm of [4] to supply the minimum decomposition.

6. 2-Castability

Another way to create a statue is to lie two mold pieces on their sides, fill the pieces with cement then, after the cement hardens, pull off the mold and glue the two pieces together on their flat sides. In two dimensions the above intuition implies that we should classify a polygon as 2-castable if it is 2-moldable with mold pieces $[p \cdots q]$ and $[q \cdots p]$ and the segment $\overline{pq}$ stays strictly inside the polygon. More formally

Definition. A polygon P is **2-castable** if $\exists p, q \in \partial P$ such that $[p \cdots q]$ and $[q \cdots p]$ are removable and $\overline{pq} - \{p, q\} \subset P^\circ$.

Definition. For $p, q \in P$ we say that p sees q whenever $\overline{pq} - \{p, q\} \subset P^\circ$.

It should be noted that while any 2-moldable polygon has a 2-mold which is determined by a pair of vertices, the same can't be said 2-castable polygons. That is, there are 2-castable polygons for which no pair of vertices determine a 2-cast.

6.1. Approach

Our purpose here is to establish an $O(n \log n)$ time and $O(n)$ space algorithm which decides the 2-castability of a polygon. Briefly, the algorithm proceeds by walking 2 chains around P . At each main step of the algorithm, it is determined whether the 2 chains determine a 2-mold for the polygon (ie. they in some way decompose the polygon into a pair of monotone chains). If this is the case then the algorithm determines whether the chains (in some way) determine a 2-cast. Determining the 2-castability of the polygon at this stage amounts to determining the visibility status of certain sets of points on the boundary of the polygon. To this end, the algorithm maintains a pair of convex hulls associated with the chains and ascertains the visibility status of the points in questions by determining the intersection status of the convex hulls.

6.2. Chain definition and 2-moldability

A chain $[p \cdots q]$ is *backward maximally monotone* if it is monotone but any extension at the p end produces a non-monotone chain. For a given polygon P , the array $B[]$ satisfies $[v_{B[i]} \cdots v_i]$ is a backwards maximal monotone chain. We will refer to $[v_{B[j]} \cdots v_j]$ and $[v_i \cdots v_{C[i]}]$ as *the back chain* at v_j and *the forward chain* at v_i respectively.

Given a polygon $P = \langle v_0, \dots, v_{n-1} \rangle$, we can reverse the roles of left and right and run algorithm *Compute C[]* on the sequence $\langle v_{n-1}, v_{n-2}, \dots, v_0 \rangle$. The result of such a process is the array $B[]$. The time and space bounds remain unchanged.

We characterize 2-Moldability in terms of the chains defined above.

Lemma 10. $\forall p, q \in \partial P$ we have

p, q determine a 2-mold for $P \Leftrightarrow$

$\exists i \in \mathbb{Z}_n$ such that at least one of the following holds

$$p \in \overline{v_i v_{i+1}} - \{v_i, v_{i+1}\}, \quad \partial P = [v_i \cdots v_{C[i]}] \cup [v_{B[i+1]} \cdots v_{i+1}],$$

$$q \in [v_{B[i+1]} \cdots v_{C[i]}]$$

$$\text{or } p = v_i, \quad \partial P = [v_i \cdots v_{C[i]}] \cup [v_{B[i]} \cdots v_i], \quad q \in [v_{B[i]} \cdots v_{C[i]}]$$

Proof. We will establish the result in the case $p = v_i$, the other case has a similar proof.

$\Rightarrow$: Assume p, q determines a 2-mold for P and $p = v_i$ is a vertex of P . Since $[p \cdots q]$

is monotone we have $[p \cdots q] \subseteq [(p = v_i) \cdots v_{C[i]}]$. Similarly $[q \cdots p] \subseteq [v_{B[i]} \cdots v_i]$ so

$$\partial P = [p \cdots q] \cup [q \cdots p] \subseteq [v_i \cdots v_{C[i]}] \cup [v_{B[i]} \cdots v_i] \subseteq \partial P$$

and $\partial P = [v_i \cdots v_{C[i]}] \cup [v_{B[i]} \cdots v_i]$. Now $\partial P - \{v_i\}$ is connected so

$$[v_{B[i]} \cdots v_i] \cap (v_i \cdots v_{C[i]}) \neq \emptyset$$

and in fact

$$[v_{B[i]} \cdots v_i] \cap (v_i \cdots v_{C[i]}) = [v_{B[i]} \cdots v_{C[i]}].$$

Note that $q \in [v_{B[i]} \cdots v_i]$ and $q \in (v_i \cdots v_{C[i]})$ so $q \in [v_{B[i]} \cdots v_{C[i]}]$.

$\Leftarrow$: Assume

$$\partial P = [v_i \cdots v_{C[i]}] \cup [v_{B[i]} \cdots v_i], \quad p = v_i, q \in [v_{B[i]} \cdots v_{C[i]}]$$

then, as before, $\partial P - \{v_i\}$ is connected therefore $[v_{B[i]} \cdots v_i] \cap (v_i \cdots v_{C[i]})$ is non-empty and in fact equals $[v_{B[i]} \cdots v_{C[i]}]$. Therefore $q \in [v_{B[i]} \cdots v_i]$ and $q \in (v_i \cdots v_{C[i]})$ from which we conclude $[q \cdots p]$ and $[p \cdots q]$ are monotone, hence P is 2-moldable with a 2-mold determined by p, q . $\square$

This Lemma tells us that we can completely characterize the set of points which determine a 2-mold for a polygon by the following linear time algorithm.

Algorithm: *Characterize Polygon*

Compute $B[]$

Computer $C[]$

for $i \leftarrow 0$ to $n - 1$

for $k \leftarrow 0$ to 1

$j \leftarrow i + k \bmod n$

if $\partial P = [v_i \cdots v_{C[i]}] \cup [v_{B[j]} \cdots v_j]$ **then**

 all pairs (p, q) with $p \in [v_i \cdots v_j]$, $q \in [v_{B[j]} \cdots v_{C[j]}]$ determine a 2-mold for P

end if

end for

end for

6.3. Determining visibility

We can use the above algorithm for determining 2-castability if we can determine whether any points in $[v_i \cdots v_j]$ see any points in $[v_{B[j]} \cdots v_{C[j]}]$. To accomplish this, assume we have the convex hulls of $[v_{C[i]} \cdots v_i]$ and $[v_j \cdots v_{B[j]}]$ (referred to as CH_i and CH_j respectively). With CH_i and CH_j in hand, we can determine visibility between points of $[v_i \cdots v_j]$ and points of $[v_{B[j]} \cdots v_{C[j]}]$ by determining the intersection status of CH_i and CH_j . The relationship between intersection and visibility will depend on whether $i = j$ or $j = i + 1$ and whether $[v_{B[j]} \cdots v_{C[j]}]$ is a single point or

not. The following lemma establishes the relationship in the case $i = j$ and $[v_{B[j]} \cdots v_{C[i]}]$ is not a single point. Of course, we are assuming $[v_{B[j]} \cdots v_{C[i]}]$ has the proper sense since we will be testing visibility only if the forward chain at i and the back chain at j cover the boundary of P .

Lemma 11. *Assume $i = j$ (i.e. $[v_i \cdots v_j] = \{v_i\}$) and $(v_{B[i]} \cdots v_i) \cap (v_i \cdots v_{C[i]}) \neq \emptyset$ then CH_i and CH_j intersect precisely at v_i if and only if some point of $(v_{B[j]} \cdots v_{C[i]})$ sees v_i .*

Proof. $\Rightarrow$: Consider a line L through v_i which places CH_i and CH_j in different half planes. Since

$$\partial P - \{v_i\} = (v_i \cdots v_{B[j]}) \cup (v_{B[j]} \cdots v_{C[i]}) \cup [v_{C[i]} \cdots v_i]$$

is connected we must have $(v_i \cdots v_{B[j]}) \cup (v_{B[j]} \cdots v_{C[i]}) \cup [v_{C[i]} \cdots v_i] \cap L \neq \emptyset$. Now since L separates CH_i and CH_j we have $(v_i \cdots v_{B[j]}) \cap L = [v_{C[i]} \cdots v_i] \cap L = \emptyset$ so $(v_{B[j]} \cdots v_{C[i]}) \cap L \neq \emptyset$.

$\Leftarrow$: Assume $\exists q \in (v_{B[j]} \cdots v_{C[i]})$ with $\overline{v_i q} - \{v_i, q\} \subset P^\circ$, we must show that CH_i and CH_j do not intersect except at v_i . Notice that it is sufficient to display a line L which separates the chains $(v_i \cdots v_{B[j]})$ and $[v_{C[i]} \cdots v_i]$. Let $L = \overline{v_i q}$, $z \in L - \overline{v_i q}$, then either $q \in \overline{v_i z}$ or $v_i \in \overline{q z}$. In the first case, if $z \in \partial P$ then any non-trivial parallel projection of L onto any line L' places the image of q between the image of v_i and the image of z contradicting the monotonicity of $[v_i \cdots q]$ or $[q \cdots v_i]$ so no such z exists. The second case is handled similarly. To obtain our result, notice that an application of the Jordan Curve Theorem places the chains $(v_i \cdots q)$ and $(q \cdots v_i)$ on opposite sides of L . $\square$

Lemma 12. *Let $j \in \mathbb{Z}_n^{i, i+1}$, then if $v_{B[j]}$ or $v_{C[i]}$ sees v_i then so does some point of $(v_{B[j]} \cdots v_{C[i]})$.*

Together, these two Lemmas tell us that in order to establish if v_i sees any point of $[v_{B[j]} \cdots v_{C[i]}]$, we must simply check whether CH_i and CH_j intersect just at v_i .

We offer, without proof, the three remaining properties relating visibility to hull intersection.

Lemma 13. (1) $j = i$ and $[v_{B[j]} \cdots v_{C[i]}]$ is a single point, then v_i sees $v_{C[i]} \Leftrightarrow CH_i$ and CH_j both have the edge $\overline{v_i v_{C[i]}}$ but with opposite orientations.

(2) $j = i + 1 \bmod n$ and $[v_{B[j]} \cdots v_{C[i]}]$ is not a single point, then some point of $[v_i \cdots v_j]$ sees some point of $[v_{B[j]} \cdots v_{C[i]}] \Leftrightarrow CH_i$ and CH_j do not intersect.

(3) $j = i + 1 \bmod n$ and $[v_{B[j]} \cdots v_{C[i]}]$ is a single point, then some point of $[v_i \cdots v_j]$ sees $v_{C[i]} \Leftrightarrow CH_i$ and CH_j intersect precisely at $v_{C[i]}$.

6.4. Background algorithms

One way to arrive at an efficient algorithm for determining 2-Castability (suggested by the last section) is to modify the *Characterize Polygon* algorithm to permit the CH_i , CH_j intersection tests. Of course, as i and j advance, we must continually make the hulls CH_i , CH_j available for such tests. This section describes efficient algorithms for maintaining the hulls and performing the intersection tests.

[3] provides us with an amortized $O(n \log n)$ time and $O(n)$ space algorithm for the dynamic maintenance of a convex hull. Its input consists of a sequence of $O(n)$ possibly intermixed insert, delete and test operations. At each stage of the algorithm a structure representing the current convex hull is either updated or queried. The structure supports logarithmic search queries along the current hull. That is, between each add and delete operation we can determine any feature of the convex hull which depends upon a binary search of the points which determine the hull.

[2] provides us with an algorithm which, given two convex hulls, determines in $O(\log n)$ time, whether the hulls intersect. The algorithm accomplishes this through two binary searches, one on each hull.

Notation. An add or delete operation is one of $\text{add}_p^Q, \text{del}_p^Q$ where $\text{add}_p^Q, \text{del}_p^Q$ means add point p to the convex hull Q and delete point p from hull Q respectively. $\text{test} \cap$ represents testing the intersection of the current hulls.

The above algorithms allow us to compute the sequence $\langle \text{op}_1, \dots, \text{op}_n \mid \text{op}_i \in \{\text{add}, \text{del}, \text{test} \cap\} \rangle$ in $O(n \log n)$ time and space. That is, we can compute any interleaved sequence of adds, deletes and intersection tests of two hulls in time $O(n \log n)$ and space $O(n)$ provided the add-delete-test sequence is determined before execution.

6.5. The modifications (putting it all together)

When $j = i$, testing whether $B[j] \in \mathbb{Z}_n^{j+1, C[i]}$ determines if there is any $q \in \partial P$ such that v_i and q determine a 2-mold. If this is the case, then *all* such q will in fact be inside $[v_{B[j]} \cdots v_{C[i]}]$. The kind of intersection test we will carry out depends on the type of chain formed by $[v_{B[j]} \cdots v_{C[i]}]$.

If the chain is a single point then we want to determine if the pair of points $v_i, v_{C[i]}$ determine a 2-cast. The pair will determine a 2-cast if and only if CH_i and CH_j intersect only on the common edge $\overline{v_i v_{C[i]}}$. This will be the case if and only if both hulls contain the edge $\overline{v_i v_{C[i]}}$ but with opposite orientations. In this case the interior of the hulls and all other edges are on opposite sides of the line determined by the common edge vertices. The algorithm of [2] admits such a test in $O(\log n)$ time.

If the chain is not a single point then we want to determine whether there is any $q \in [v_{B[j]} \cdots v_{C[i]}]$ so that the segment $\overline{v_i q}$ remains inside the polygon. Equivalently, we test whether v_i is a vertex of CH_i and CH_j and check the neighboring vertices to see

if in fact the hulls intersect precisely at this point. To be exact, we check if the infinite wedges based on v_i determined by v_i and its 4 neighbors, the immediate successors and predecessors of v_i on CH_i and CH_j , intersect. If v_i is not a vertex of either hull or the wedges intersect then the two hulls intersect and there is no 2-cast determined by v_i and any point in $[v_{B[j]} \cdots v_{C[i]}]$.

The corresponding hull operations (prior to changing j) amount to the single delete $\text{del}_{v_j}^{CH_j}$ and the possibly 0 or more adds $\text{add}_{v_{B[j]+1}}^{CH_j} \cdots \text{add}_{v_{B[j+1]}}^{CH_j}$.

As above, we break the case $j = i + 1 \bmod n$ into two subcases. If the back and forward chains overlap at a single point then we determine the 2-castability status of the polygon in much the same way as the chain overlap subcase in the $j = i$ case above. Finally if the back and forward chains overlap then we determine 2-castability by determining if CH_i and CH_j intersect. They don't overlap precisely when the polygon is 2-castable. We advance by incrementing i , executing the $\text{add}_{v_i}^{CH_i}$ and the possibly 0 or more deletes $\text{del}_{v_{C[i-1]}}^{CH_i} \cdots \text{del}_{v_{C[i]}}^{CH_i}$. We are now back in the $i = j$ case and we continue.

It should be clear that after advancing n times ($j = i, j = i + 1$) we have returned to our starting position and so have checked the entire polygon. An amortized analysis shows that each vertex appears at most twice in either convex hull so that there are $O(n)$ adds and dels. There are also only $O(n)$ test $\cap$ s since we perform some constant number for each i . Consequently the algorithm is in $O(n \log n)$. The $O(n)$ space bounds apply since the underlying structures require linear space provided we can schedule the test operations before running the algorithm. We can either perform all four intersection queries on the structure at each step of the algorithm or we can 'simulate' the algorithm once to determine the test scheduling and run the full algorithm later. In either case we have the same result.

Theorem 14. *There is a $O(n \log n)$ time, $O(n)$ space algorithm which determines if an input polygon is 2-castable.*

Without too much more effort, additional $O(n)$ time, we can determine a pair of points which determine a 2-Cast (provided the polygon is 2-castable).

7. Future work

Obvious extensions include determination of a polygon's k -moldability/ k -castability in dimensions 2 and 3. Of particular interest is the determination of 2-moldability in 3 dimensions. Alternatively, variations on the moldability/castability theme which allow rotation as well as transformation should be investigated.

References

- [1] D. Rappaport and A. Rosenbloom, Moldable and castable polygons in: Proceedings of the Fourth Canadian Conference on Computational Geometry (1992).
- [2] B. Chazelle and D.P. Dobkin, Intersection of convex objects in two and three dimensions, *J. Assoc. Comput. Mach.* 34 (1987) 1–27.
- [3] J. Hershberger and S. Suri, Offline maintenance of planar configurations, in: SODA 91 (1991).
- [4] C.C. Lee and D.T. Lee, On a circle cover minimization problem, *Inform. Process. Lett.* 18 (1984) 109–115.
- [5] F.P. Preparata and M.I. Shamos, *Computational Geometry an Introduction* (Springer, Berlin, 1985).
- [6] F.P. Preparata and K.J. Supowit, Testing a simple polygon for monotonicity, *Inform. Process. Lett.* 12 (1981) 161–164.
- [7] A. Rosenbloom, Moldable and castable polygons, Master's Thesis, Queen's University, 1992.
- [8] R. Swaminathan, V. Chandru and V.T. Rajan, Monotone pieces of chains, in: Proceedings of The Second Canadian Conference on Computational Geometry (1990).
- [9] R. Swaminathan, V. Chandru and V.T. Rajan, Monotone pieces of chains, *ORSA J. Comput.* (1992).